



HEART DISEASE

[Document subtitle]



[DATE]

[COMPANY NAME]

[Company address]

1. Introduction

Heart disease remains one of the leading causes of mortality worldwide. Predicting the risk of heart disease at an early stage can help in effective treatment and preventive healthcare.

This project implements a **comprehensive machine learning pipeline** on the **UCI Heart Disease dataset**. The pipeline includes **data preprocessing, dimensionality reduction, feature selection, supervised and unsupervised learning, hyperparameter tuning, and model export**.

2. Dataset Description

- **Source:** [UCI Machine Learning Repository – Heart Disease](#)
- **Format:** CSV (heart_disease.csv)
- **Samples:** 303 patients
- **Features:** 13 attributes + target (binary classification)
- **Target variable:**
 - 1 → Heart disease present
 - 0 → No heart disease

Features

- age – Age of patient
- sex – Gender (1 = male, 0 = female)
- cp – Chest pain type (0–3)
- trestbps – Resting blood pressure
- chol – Serum cholesterol (mg/dl)
- fbs – Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)
- restecg – Resting ECG results
- thalach – Maximum heart rate achieved
- exang – Exercise induced angina (1 = yes, 0 = no)
- oldpeak – Depression induced by exercise relative to rest

- slope – Slope of peak exercise ST segment
 - ca – Number of major vessels (0–3) colored by fluoroscopy
 - thal – Thalassemia (0–3)
 - target – Output class (0 = no disease, 1 = disease)
-

3. Data Preprocessing & Cleaning

Steps:

1. Load the dataset.
2. Remove missing values.
3. Encode categorical features.
4. Scale numerical features.

Code:

```
import pandas as pd

from sklearn.preprocessing import StandardScaler

# Load dataset

df = pd.read_csv("data/heart_disease.csv")

df = df.dropna() # remove missing values

df = pd.get_dummies(df, drop_first=True)

# Scale features

scaler = StandardScaler()

X = scaler.fit_transform(df.drop("target", axis=1))

y = df["target"]
```

4. Dimensionality Reduction – PCA

Steps:

1. Apply PCA to reduce features.

2. Plot explained variance.

Code:

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

pca = PCA()
X_pca = pca.fit_transform(X)

plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("PCA Variance Explained")
plt.show()
```

5. Feature Selection

Steps:

- Recursive Feature Elimination (RFE)
- Chi-Square test
- Random Forest importance

Code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE, chi2, SelectKBest
from sklearn.ensemble import RandomForestClassifier

# RFE
model = LogisticRegression(max_iter=1000)
rfe = RFE(model, n_features_to_select=10)
```

```
rfe.fit(X, y)
```

```
# Chi-Square
```

```
chi2_selector = SelectKBest(chi2, k=10)
```

```
chi2_selector.fit(X, y)
```

```
# Random Forest Importance
```

```
rf = RandomForestClassifier(random_state=42)
```

```
rf.fit(X, y)
```

```
print("Feature Importances:", rf.feature_importances_)
```

6. Supervised Learning

Models:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)

Code:

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import classification_report, roc_auc_score
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.svm import SVC
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
models = {
```

```
    "Logistic Regression": LogisticRegression(max_iter=1000),
```

```
    "Decision Tree": DecisionTreeClassifier(),
```

```
"Random Forest": RandomForestClassifier(),  
"SVM": SVC(probability=True)  
}
```

```
for name, clf in models.items():  
    clf.fit(X_train, y_train)  
    y_pred = clf.predict(X_test)  
    print(f"\n{name} Results:")  
    print(classification_report(y_test, y_pred))  
    if hasattr(clf, "predict_proba"):  
        y_proba = clf.predict_proba(X_test)[:, 1]  
        print("AUC:", roc_auc_score(y_test, y_proba))
```

7. Unsupervised Learning

Steps:

- KMeans clustering
- Hierarchical clustering (dendrogram)

Code:

```
from sklearn.cluster import KMeans  
  
import scipy.cluster.hierarchy as sch  
  
# KMeans  
kmeans = KMeans(n_clusters=2, random_state=42)  
clusters = kmeans.fit_predict(X)  
  
# Hierarchical Clustering  
plt.figure()  
sch.dendrogram(sch.linkage(X, method='ward'))
```

```
plt.title("Hierarchical Clustering Dendrogram")  
plt.show()
```

8. Hyperparameter Tuning

Steps:

- Use GridSearchCV for Random Forest.

Code:

```
from sklearn.model_selection import GridSearchCV  
  
param_grid = {  
    "n_estimators": [50, 100, 200],  
    "max_depth": [None, 5, 10],  
    "min_samples_split": [2, 5, 10]  
}  
  
grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=3,  
                    scoring="accuracy")  
  
grid.fit(X_train, y_train)  
  
print("Best Parameters:", grid.best_params_)
```

9. Model Export & Deployment

Steps:

- Save pipeline (scaler + model).
- Export as .pkl.

Code:

```
from sklearn.pipeline import Pipeline  
import joblib
```

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("rf", grid.best_estimator_)
])

pipeline.fit(X_train, y_train)

joblib.dump(pipeline, "models/final_pipeline.pkl")

print("✅ Pipeline saved as models/final_pipeline.pkl")
```

10. GitHub Repository

The full project (datasets, notebooks, models, requirements, and documentation) is available here:

👉 **GitHub Link:** [HTTPS://GITHUB.COM/OMARSAHER/HEART-DISEASE](https://github.com/OMARSAHER/HEART-DISEASE)

11. Conclusion

This project successfully implemented a full **machine learning pipeline** for heart disease prediction. Logistic Regression and Random Forest provided strong performance, while unsupervised clustering revealed meaningful patient groupings. The final model was exported as a .pkl file for reproducibility and deployment.
